Template → Stencil/ So it lets us write one piece of code that works for all datatypes

→ Benifits

1) Avoids writing the same code again and again
2) make the code more flexible

(Function template)

```
template <typename T>
Tadd (Ta, Tb) {
    return a+b
}
```

this one function works for all float, int..

Function overloading

→ writing many functions with same name but diff parameters

```
Intadd (Inta, Int b)
Int add (float b, Int y)
Int add (double a, double b)
```

typename and class means the same thing.

Syntax

template < para >

template<class type> or

template <typename Type>

typename T

?

T is an empty box

(later you will fill the box with a

↓

(Placeholder) datatype

Int, float

depends on what you pass in function

real datatype)

```
template <typename T>          <- header
T max (T a, T b)
{                              <- code
    if (a > b)
        return a;
    else
        return b;
}
```

```
template <typename T>
?? cout << "hello";
T func (T x, T y)
{
    return x+y;
}
```

```
template <typename T>
T max (T a, T b)
```

max() returns a value so function type is T.

```
template <typename T>
void Swap (T a, T b)
```

→ doesn't return anything

---

calling of function templates

```
max (55, 80)
max ('5', '7')
max (7.23, 6)
```

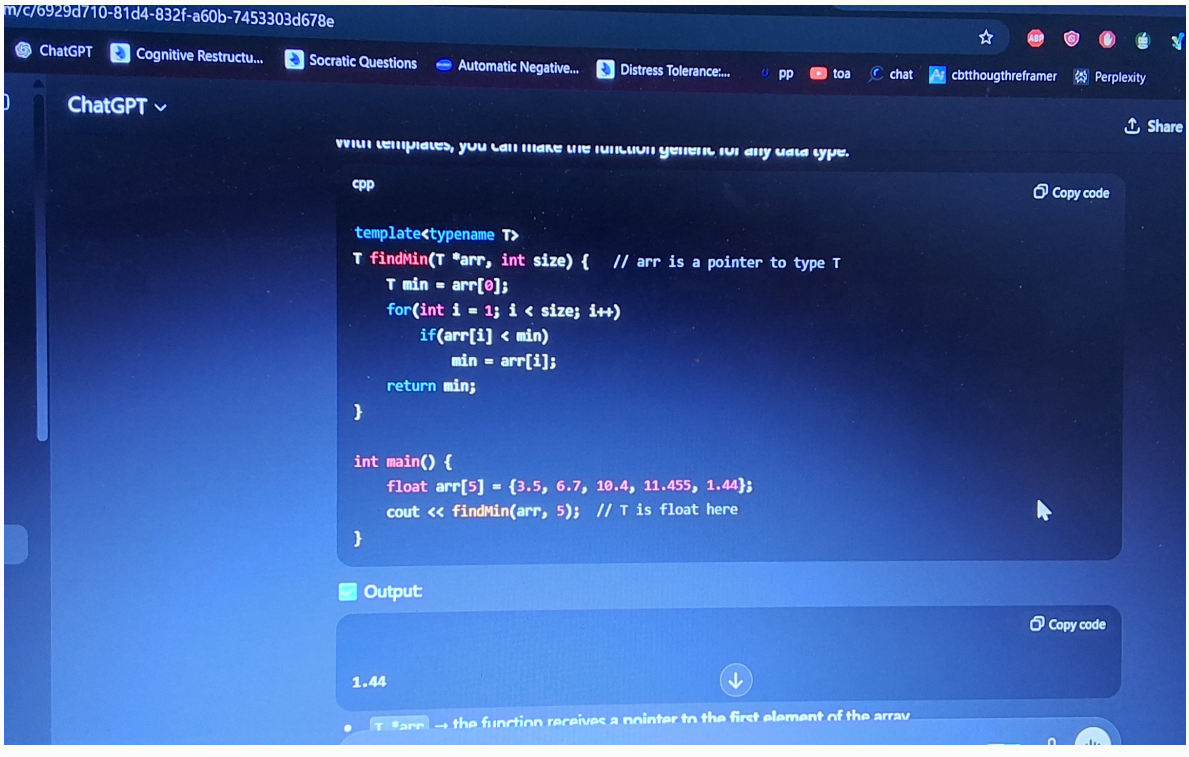
} the compiler generates seperate copies for each d.t called

retun a,b → returns only b

note: If you want to swap diff types (like INT ↔ float) you need two template para.

```
template < typename T1, typename T2>
void Swapdiff ( T1 &a, T2 &b)
{
    T1 temp = a;
    a = b
    b = temp;
}
```

finding Max element in a array using templates function.



T arr[] Same as T *arr

Templates with multiple type parameters.

```
template < typename T1, typename T2 >
    void print ( T1 a, T2 b)
        {
            cout << " first is " << a << endl;
            cout << " Second is " << b << endl;
        }
```

Types of errors:

1) compile-time errors aka Syntax errors: e.g missing Σ3
2) logical errors → wrong output
3) runtime error (exception) → errors happen while the program is still running
   e.g) insufficient memory

(Exception handling)

→ we put the code that might cause error inside a try Block.

```
try {
    int x = a / b
    }
```

b might be 0 → error

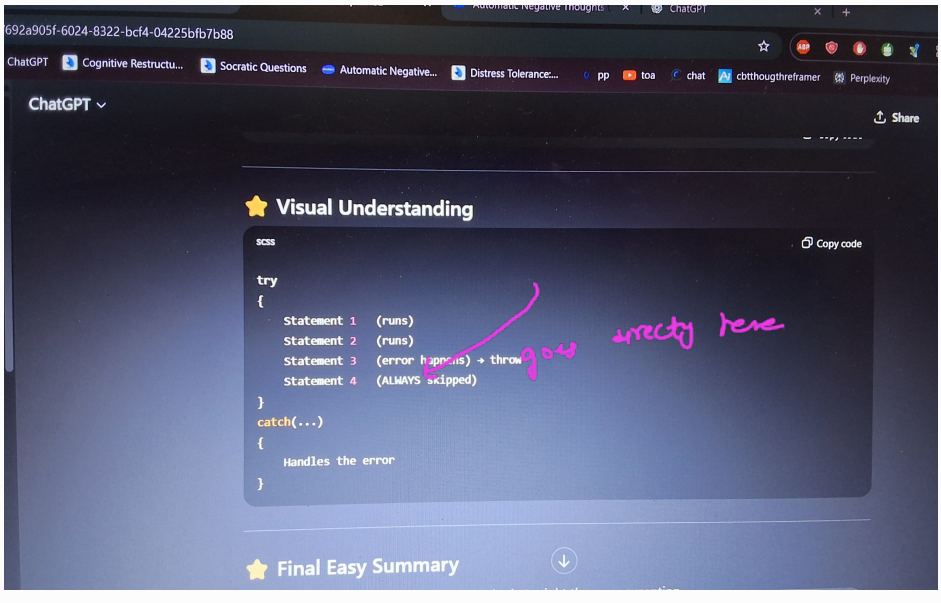
if error happens

```
throw "Division by zero";
```

catch Block receives the throw exception and handles it.

```
catch (char msg) {
    cout<< msg;
}
```

```
try {
    int x = 5;      // x exist only inside try block → local
}
```




```
throw <value>
    ↓
int, float etc

throw 5;
throw "divide by 0";
    → const char*

throw string("reboot");
    → string
```

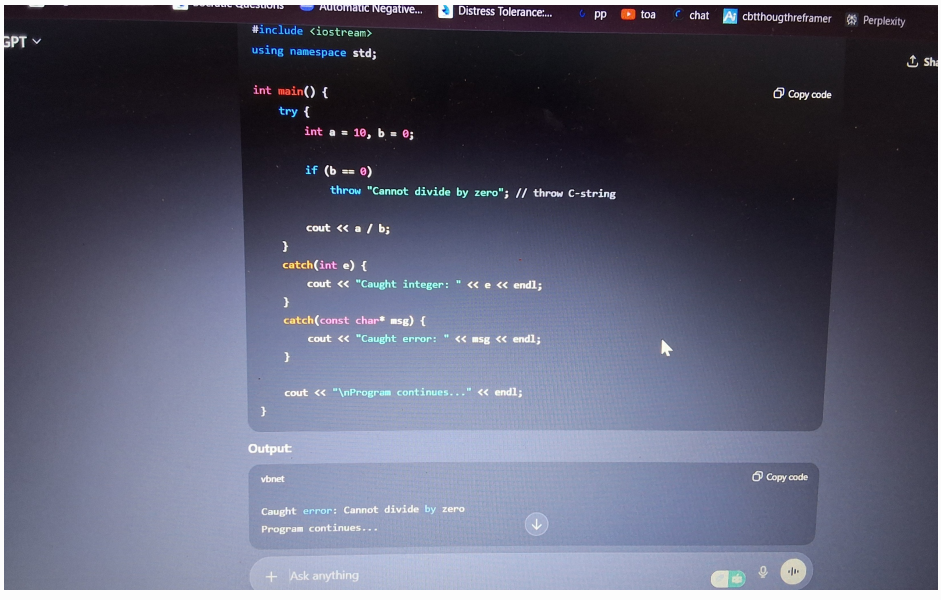
```
catch (int e) {
    cout<< "caught" << e << "\n";
    throw;
}
```

→ re-throw to another catch

Note.. → Every try Block must have atleast 1 catch Block

→ We can have multiple catch Block

→ Catch Block takes only 1 argument



Exception handling in function

```
void divide (int a, int b)
{   if(b==0)
      throw b;
    cout<< "result" << a/b;
}
```

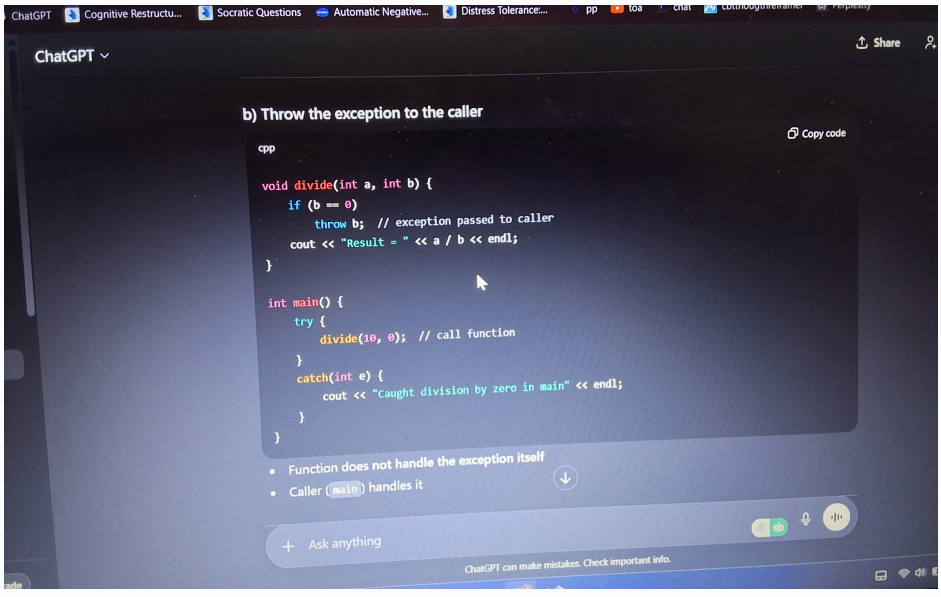
→ exception thrown inside a function

---

1) handling the exception locally.

```
void divide (int a, int b) {
    try { if (b==0)
            throw b;
          cout<< "Result" << a/b;
    }
    catch (int e) {
          cout<< "illegal division"
    }
}
```

2) Throw the exception to the caller



Stack unwinding:- main() → f1() → f2()

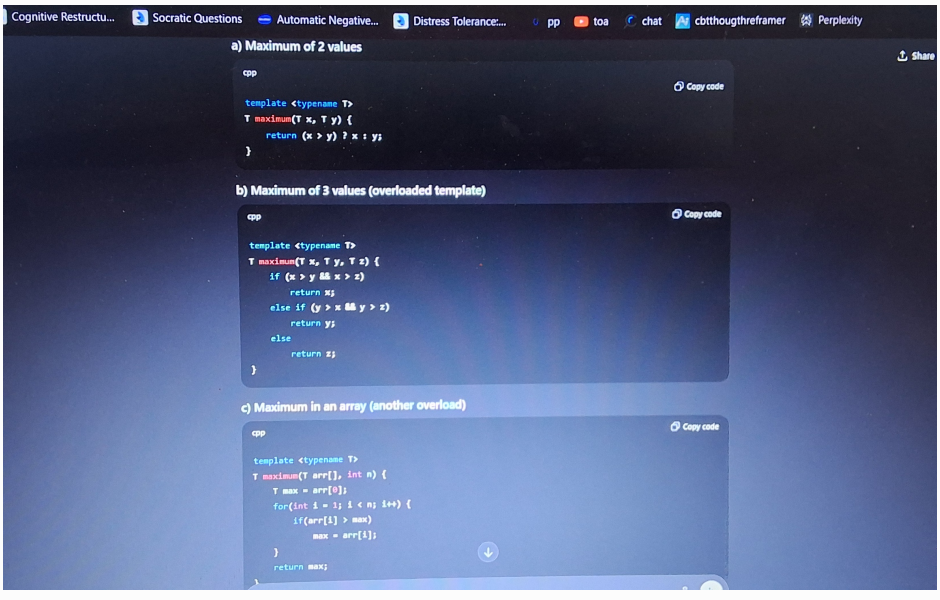
however Something goes wrong in f2() and it [throws] but doesn't have a **catch** BLOCK

So it goes to f1() to See if there is **catch**

if not then go to main().
if still no **catch** then program crashes.

(Pg-16- 23) left

Function template overloading:: → function name and return type remains the same but no of para change



(Note:: Function templates don't work in all situations)

```
template <typename T>
T max (T x, T y)
    if (x > y)
        return x
    else
        y
```

doesn't work for char arrays

```
char arr1[5] = "apple";
char arr2[5] = "zebra";
```

C-string uses pointer comparision because arrays decay into pointers when passed into a function.

```
Char letter = 'A';

void printChar (char c){
        cout<<c;
}
```

C-String (Char array)




```
Char name[] = "Ali";
```




```
void printChar( const char Str[]){
        cout<<Str;
}
```

→ passed address of the first char

```
String s = "hello";
void printString( const String &L)
{   cout<< L;
}
```

→ passed by const ref.
No copy and cannot modify the orginal

```
void printString( String L )
```

→ full copy

```
void printString( String &L )
```

→ No copy and can modify the orignal as there is no const

(Function template Specialization) → used because one datatype needs different logic

Steps to specialize a template function

1) Add an empty template header

template<> → now this is a special version

2) char * max<char *>(char * x, char * y)

this version only works for char arrays.

char * → character array

const char * → character arrays

char array[] = "hello";

char * p = array;

p[0] = 'h';

const char *s = "hello";

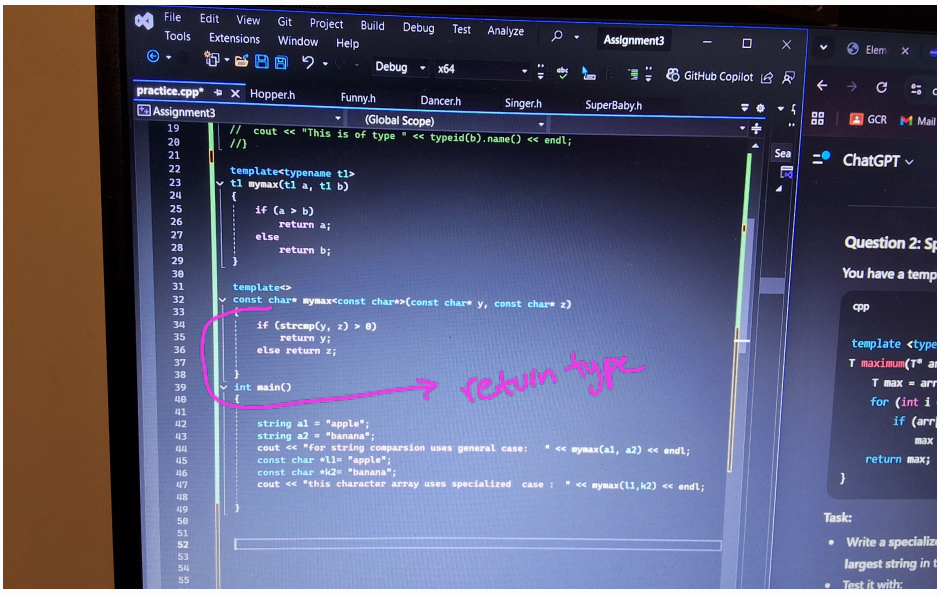
↓ pointer to read only character array

Strcmp(x, y)

x > y returns value > 0

x < y < 0

x = y 0



role Specialization only works if general template is available